



CSS Animations

Create dynamic, engaging animations with CSS keyframes and interactive demos

What are CSS Animations?

CSS Animations allow you to create complex, multi-step animations that can loop, pause, and control timing with precision. Unlike transitions, animations use keyframes to define specific points in the animation timeline.

Animations are perfect for continuous effects, loading spinners, hero animations, and complex sequences that transitions alone cannot achieve.

Browser Support: All modern browsers support CSS Animations. Use `-webkit-` prefix for older Safari and mobile browsers for optimal compatibility.

Core Animation Concepts

1. @keyframes Rule

Defines the animation steps and their properties at specific percentages or keywords (from/to).

Syntax:

```
@keyframes animationName {
  0% { /* starting styles */ }
  50% { /* middle styles */ }
  100% { /* ending styles */ }
}
```

Alternative Syntax:

```
@keyframes animationName {
  from { /* same as 0% */ }
  to { /* same as 100% */ }
}
```

2. animation-name

Specifies the name of the @keyframes animation to apply.

Syntax: animation-name: keyframeName;
Example: animation-name: spin, bounce;

3. animation-duration

Defines how long the animation takes to complete one cycle.

Syntax: animation-duration: time;
Units: seconds (s) or milliseconds (ms)
Default: 0s

4. animation-timing-function

Controls the speed curve of the animation (same options as transitions).

Values: linear, ease, ease-in, ease-out, ease-in-out, steps(), cubic-bezier()

5. animation-delay

Specifies when the animation starts after being applied.

Syntax: animation-delay: time;
Units: seconds (s) or milliseconds (ms)
Default: 0s

6. animation-iteration-count

Determines how many times the animation repeats.

Values:

- number - Specific number of cycles
- infinite - Repeats forever
- 0.5 - Play only halfway (fractional values allowed)

7. animation-direction

Specifies whether the animation plays forward, backward, or alternates.

Values:

- normal - Forward each cycle (default)
- reverse - Backward each cycle
- alternate - Forward then backward
- alternate-reverse - Backward then forward

8. animation-fill-mode

Controls styles applied before/after animation plays.

Values:

- none - No styles applied before/after (default)
- forwards - Keeps final keyframe styles
- backwards - Applies first keyframe styles immediately
- both - Applies both

9. animation-play-state

Pauses or plays the animation.

Values: running (default) | paused

10. animation (Shorthand)

Combines all animation properties in one declaration.

Syntax: animation: [name] [duration] [timing] [delay] [count] [direction] [fill-mode] [play-state];

Example:
animation: spin 2s linear infinite;
animation: bounce 1s ease-in-out 0.5s 3 alternate;



Interactive Animation Demos



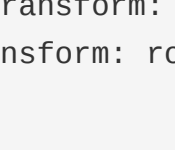
Spin
linear, infinite



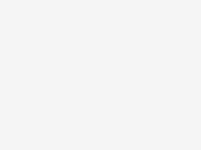
Bounce
ease-in-out, infinite



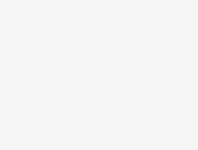
Pulse
ease-in-out, infinite



Slide In
ease-in-out, infinite



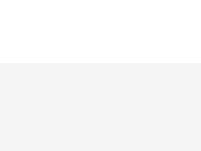
Shake
0.5s, infinite



Glow
ease-in-out, infinite



Fade
ease-in-out, infinite

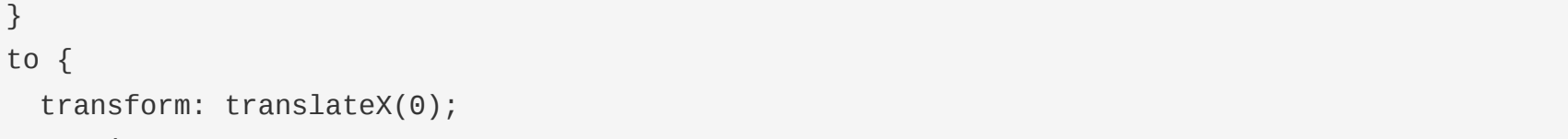


Scale
ease-in-out, infinite

Additional Animation Demos



Loading Spinner



Loading Progress Bar

ANIMA

Text Bounce Animation

Common Use Cases & Patterns

Infinite Spinning Animation (Loading)

```
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}

.spinner {
  animation: spin 1s linear infinite;
}
```

Fade In Animation

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

.element {
  animation: fadeIn 1s ease-in;
  animation-fill-mode: forwards;
}
```

Slide In From Left

```
@keyframes slideInLeft {
  from {
    transform: translateX(-100%);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}

.element {
  animation: slideInLeft 0.5s ease-out;
}
```

Bounce Animation

```
@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  25% { transform: translateY(-20px); }
  50% { transform: translateY(0); }
  75% { transform: translateY(-10px); }
}

.element {
  animation: bounce 1s ease-in-out infinite;
}
```

Staggered Animation (Multiple Elements)

```
.item:nth-child(1) { animation-delay: 0s; }
.item:nth-child(2) { animation-delay: 0.1s; }
.item:nth-child(3) { animation-delay: 0.2s; }
.item:nth-child(4) { animation-delay: 0.3s; }
```

Pause Animation on Hover

```
.animated-element:hover {
  animation-play-state: paused;
}
```

Best Practices & Tips

- Performance First:** Animate `transform` and `opacity`. Avoid animating layout properties (width, height, position).
- Use Will-Change:** Add `will-change: transform;` to hint the browser for optimization.
- GPU Acceleration:** Add `transform: translate3d(0, 0, 0);` to force GPU rendering.
- Animation Duration:** Keep animations between 0.2s - 1s for UI feedback. Longer for hero animations.
- Infinite vs Count:** Use `infinite` sparingly. Most animations should have a defined iteration count.
- Fill-Mode Strategy:** Use `forwards` to maintain final state, `backwards` for immediate start state.
- Test Reduced Motion:** Respect `prefers-reduced-motion` for accessibility.
- Timing Functions:** `ease-out` for entrances, `ease-in` for exits, `linear` for continuous motion.
- Keyframe Percentages:** Use specific percentages (25%, 75%) for smoother, more complex animations.

Animations vs Transitions: When to Use Each

Feature	Animations	Transitions
Trigger	Automatic (applied to element)	Event-based (hover, click, class change)
Control	Full control via keyframes	Single state change
Repeat	Can repeat infinitely	One-time only per state change
Complexity	Multi-step complex animations	Simple smooth state changes
Use Case	Loading spinners, hero animations, continuous effects	Hover effects, focus states, interactions

Advanced Techniques

Multiple Keyframe Percentages

```
@keyframes complexAnimation {
  0%, 100% { transform: scale(1); opacity: 1; }
  25% { transform: scale(1.1); opacity: 0.8; }
  50% { transform: scale(0.9); opacity: 1; }
  75% { transform: scale(1.05); opacity: 0.9; }
}
```

Animation Play State Control

```
.element {
  animation: spin 2s linear infinite;
}

.element.paused {
  animation-play-state: paused;
}

/* Pause on hover */
.element:hover {
  animation-play-state: paused;
}
```

Direction Alternation

```
/* Plays forward then backward */
.element {
  animation: bounce 2s ease-in-out 3s alternate;
}

/* Plays backward then forward */
.element {
  animation: bounce 2s ease-in-out 3s alternate-reverse;
}
```

CSS Filters in Animations

```
@keyframes filterEffect {
  0% {
    filter: brightness(1) blur(0px) saturate(100%);
  }
  50% {
    filter: brightness(1.2) blur(2px) saturate(150%);
  }
  100% {
    filter: brightness(1) blur(0px) saturate(100%);
  }
}
```

Accessibility: Respecting Reduced Motion

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Browser Compatibility

Modern Browsers: Chrome, Firefox, Safari, Edge support CSS Animations natively without prefixes.

Older Browsers: Use `-webkit-` prefix for Safari 5-8 and older iOS/Android browsers.

Internet Explorer: IE 10+ supports CSS Animations with `-ms-` prefix (limited in older versions).

Vendor Prefixes Example

```
@-webkit-keyframes spin {
  from { -webkit-transform: rotate(0deg); }
  to { -webkit-transform: rotate(360deg); }
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
```

Performance Optimization

- Animate Transform & Opacity:** These properties don't trigger reflows. Width/height/left/top do.
- GPU Acceleration:** Use `transform: translateZ(0);` or `will-change: transform;` to enable hardware acceleration.
- Reduce Animations on Low-End Devices:** Test on mobile and disable heavy animations if needed.
- Use CSS Animations Over JavaScript:** CSS is optimized by browsers and runs on GPU when possible.
- Limit Animation Count:** Too many simultaneous animations can cause jank. Stagger them with delays.
- Profile with DevTools:** Use Chrome DevTools Performance tab to identify bottlenecks.
- Consider Animation Complexity:** Simpler keyframes (fewer steps) = better performance.

Quick Reference Summary

Property	Purpose	Default
animation-name	Name of @keyframes animation	none
animation-duration	How long animation lasts	0s
animation-timing-function	Speed curve of animation	ease
animation-delay	Delay before animation starts	0s
animation-iteration-count	How many times animation repeats	1
animation-direction	Direction: forward, reverse, alternate	normal
animation-fill-mode	Styles before/after animation	none
animation-play-state	running or paused	running

Common Mistakes to Avoid

- Forgetting animation-duration:** Without duration, animation runs instantly (default 0s).
- Incorrect keyframe percentages:** Use 0%, 50%, 100% or from/to. Invalid percentages are ignored.
- Animation-name doesn't exist:** Check that @keyframes name matches animation-name exactly.
- Using infinite too much:** Infinite animations drain battery on mobile. Use count instead.
- Not using animation-fill-mode:** Element reverts to original state after animation ends. Use forwards/backwards.
- Animating expensive properties:** Width, height, left, top trigger reflows. Use transform instead.
- Ignoring accessibility:** Users with motion sensitivity need prefers-reduced-motion support.
- Complex animations on every page load:** Use CSS animations for hero sections, not every element.